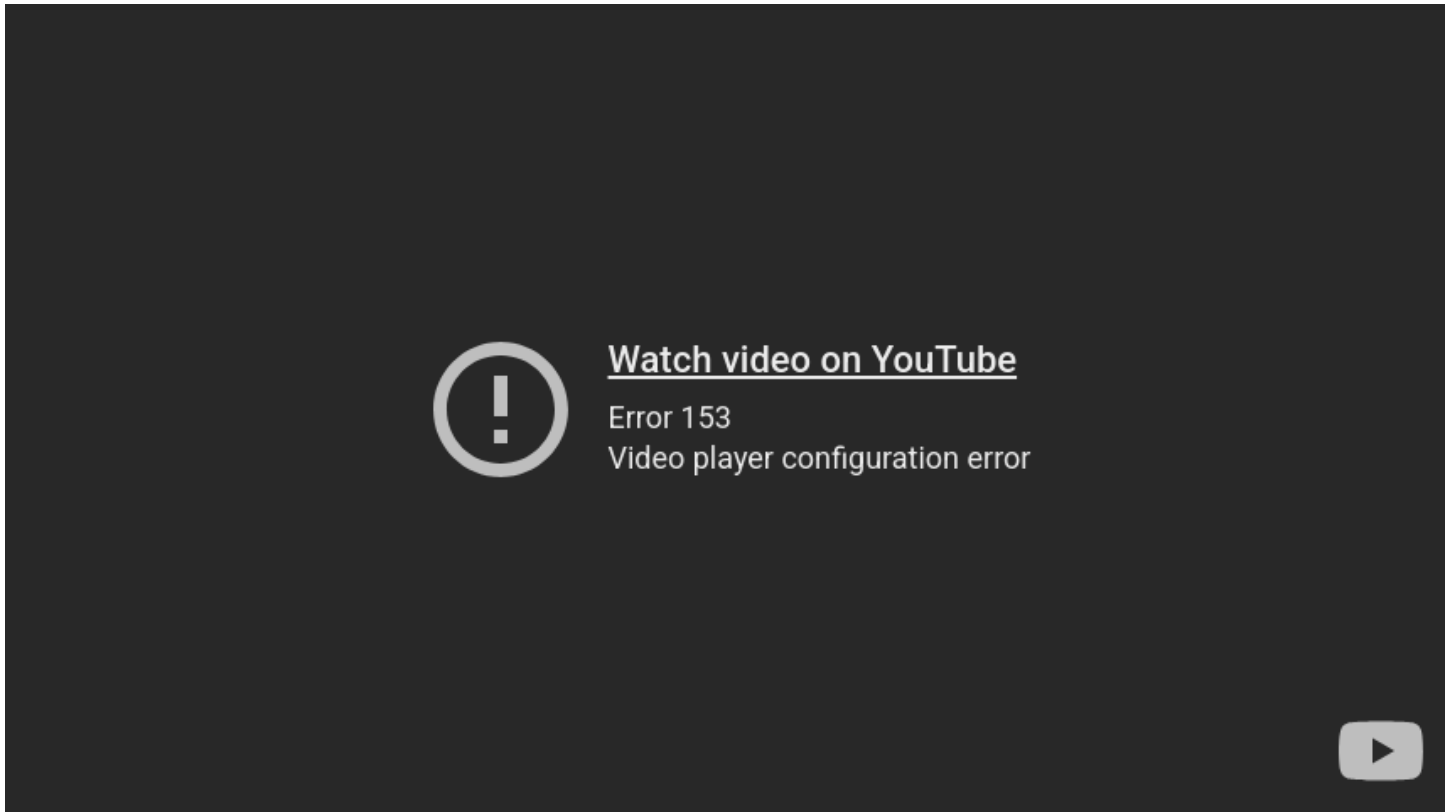


Laboratório 4 OAC Grupo A4



1.1)

Para transformar o processador Uniciclo em Pipeline, foram inseridos quatro conjuntos de registradores de barreira entre as etapas de execução. Esses registradores são responsáveis por armazenar dados de controle e dados operacionais (endereços, imediatos e valores lidos) na borda de subida do clock, permitindo que cada estágio opere de forma independente.

Abaixo está o trecho de código em Verilog que implementa a definição e a lógica de escrita desses registradores:

```
//  
=====
```

// 1. DECLARAÇÃO DOS REGISTRADORES DE PIPELINE

```
//  
=====
```

// 1.1 IF/ID (Instruction Fetch -> Instruction Decode)

// Armazena: PC (32) + Instrução (32)

reg [63:0] REG_IF_ID;

// 1.2 ID/EX (Instruction Decode -> Execute)

// Armazena:

// - Sinais de Controle: WB(3), MEM(4), EX(5)

// - Dados: PC(32), Leitura1(32), Leitura2(32), Imediato(32)

// - Instrução: Funct(4), Rd(5)

// Total: 149 bits

reg [148:0] REG_ID_EX;

// 1.3 EX/MEM (Execute -> Memory Access)

// Armazena:

// - Sinais de Controle: WB(3), MEM(4)

// - Dados: AlvoBranch(32), Zero(1), ResultadoULA(32), DadoEscrita(32), Rd(5)

// Total: 109 bits

reg [108:0] REG_EX_MEM;

// 1.4 MEM/WB (Memory Access -> Write Back)

// Armazena:

// - Sinais de Controle: WB(3)

// - Dados: DadoLidoMem(32), ResultadoULA(32), Rd(5)

// Total: 72 bits

reg [71:0] REG_MEM_WB;

```
//
```

```
=====
```

```
// 2. LÓGICA DE ATUALIZAÇÃO DOS REGISTRADORES (Síncrona na Borda de Subida)
```

```
//
```

```
=====
```

```
// Atualização do IF/ID
```

```
always @(posedge clockCPU or posedge reset) begin
```

```
    if (reset) REG_IF_ID <= 64'b0;
```

```
    else    REG_IF_ID <= {PC, wInstr_IF};
```

```
end
```

```
// Atualização do ID/EX
```

```
// Empacota sinais de controle gerados no ID e dados lidos do Banco de Registradores
```

```
always @(posedge clockCPU or posedge reset) begin
```

```
    if (reset) REG_ID_EX <= 149'b0;
```

```
    else REG_ID_EX <= {
```

```
        wRegWrite_ID, wMemToReg_ID,    // WB (3 bits)
```

```
        wMemRead_ID, wMemWrite_ID, wBranch_ID, wJump_ID, // MEM (4 bits)
```

```
        wALUSrc_ID, wALUOp_ID, wJumpReg_ID, // EX (5 bits)
```

```
        wPC_ID, wReadData1_ID, wReadData2_ID, wImm_ID, // Dados Operandos
```

```
        wInstr_ID[30], wInstr_ID[14:12], wInstr_ID[11:7] // Funct7, Funct3, Rd
```

```
    };
```

```
end
```

```
// Atualização do EX/MEM
```

```
// Empacota resultados da ULA e mantém dados de store para a Memória
```

```

always @(posedge clockCPU or posedge reset) begin

    if (reset) REG_EX_MEM <= 109'b0;

    else REG_EX_MEM <= {

        wRegWrite_EX, wMemToReg_EX,    // Propaga WB

        wMemRead_EX, wMemWrite_EX, wBranch_EX, wJump_EX, // Consome MEM

        wBranchTarget_EX, wZero_EX,    // Resultados Branch

        wALUResult, wReadData2_EX,    // Resultado ULA e Dado Store

        wRd_EX                        // Destino

    };

end

```

// Atualização do MEM/WB

// Seleciona dado lido da memória ou resultado da ULA para escrita final

```

always @(posedge clockCPU or posedge reset) begin

    if (reset) REG_MEM_WB <= 72'b0;

    else REG_MEM_WB <= {

        wRegWrite_MEM, wMemToReg_MEM,    // Consome WB

        wMemReadData_MEM,                // Dado da Memória

        wALUResult_MEM,                  // Dado da ULA

        wRd_MEM                          // Endereço de Escrita

    };

end

```

1.2)

O processador Pipeline foi implementado dividindo o caminho de dados em 5 estágios (IF, ID, EX, MEM, WB) interligados por registradores de pipeline. A arquitetura suporta o conjunto de instruções reduzido RV32I exigido (add, sub, and, or, slt, lw, sw, beq, jal, jalr, addi, lui).

`ifndef PARAM

```

`include "Parametros.v"

`endif

module Pipeline (

    input logic clockCPU, clockMem,

    input logic reset,

    output logic [31:0] PC,

    output logic [31:0] Instr,

    input logic [4:0] regin,

    output logic [31:0] regout,

    output logic [3:0] estado

);

//
=====

// 1. DEFINIÇÃO DOS REGISTRADORES DE PIPELINE

//
=====

reg [63:0] REG_IF_ID;

// WB(3) + MEM(4) + EX(5) + PC(32) + A(32) + B(32) + Imm(32) + Func(4) + Rd(5) = 149 bits

reg [148:0] REG_ID_EX;

// WB(3) + MEM(4) + BranchTarget(32) + Zero(1) + ALURes(32) + WriteData(32) + Rd(5) = 109
bits

reg [108:0] REG_EX_MEM;

// WB(3) + ReadData(32) + ALURes(32) + Rd(5) = 72 bits

reg [71:0] REG_MEM_WB;

```

```
//
```

```
=====
```

```
// 2. FIOS (WIRES) GERAIS
```

```
//
```

```
=====
```

```
// --- Sinais do Estágio WB (Feedback) ---
```

```
wire wRegWrite_WB;
```

```
wire [4:0] wRd_WB;
```

```
wire [31:0] wWriteData_WB;
```

```
// --- Estágio IF ---
```

```
wire [31:0] wPC_Next, wPC_Plus4, wInstr_IF;
```

```
// PCSrc agora será decidido pelo estágio MEM (onde validamos o Branch)
```

```
// Mas no pipeline simples, algumas decisões voltam do EX ou MEM.
```

```
// Vamos assumir decisão no MEM para evitar Hazard de controle complexo agora.
```

```
wire PCSrc;
```

```
wire [31:0] wBranchTarget_MEM; // Vem do estágio MEM
```

```
// --- Estágio ID ---
```

```
wire [31:0] wPC_ID, wInstr_ID;
```

```
wire wRegWrite_ID, wALUSrc_ID, wMemRead_ID, wMemWrite_ID, wBranch_ID, wJump_ID,  
wJumpReg_ID;
```

```
wire [1:0] wMemToReg_ID;
```

```
wire [2:0] wALUOp_ID;
```

```
wire [31:0] wReadData1_ID, wReadData2_ID, wImm_ID;
```

```

// --- Estágio EX ---

// Sinais desempacotados do REG_ID_EX

wire wRegWrite_EX, wMemRead_EX, wMemWrite_EX, wBranch_EX, wJump_EX, wALUSrc_EX,
wJumpReg_EX;

wire [1:0] wMemToReg_EX;

wire [2:0] wALUOp_EX;

wire [31:0] wPC_EX, wReadData1_EX, wReadData2_EX, wImm_EX;

wire [4:0] wRd_EX;

wire [2:0] wFunct3_EX;

wire wFunct7_EX;


// Sinais calculados no EX

wire [4:0] wALUCtrl;

wire [31:0] wALUResult, wALUInputB;

wire wZero_EX;

wire [31:0] wBranchTarget_EX;


// --- Estágio MEM ---

wire wRegWrite_MEM, wMemRead_MEM, wMemWrite_MEM, wBranch_MEM, wJump_MEM,
wZero_MEM;

wire [1:0] wMemToReg_MEM;

wire [31:0] wALUResult_MEM, wWriteDataMem_MEM;

wire [4:0] wRd_MEM;

wire [31:0] wMemReadData_MEM;

```

```
//
```

```
=====
```

```
// 3. ESTÁGIO IF (INSTRUCTION FETCH)
```

```
//
```

```
=====
```

```
assign wPC_Plus4 = PC + 4;
```

```
// Se PCSrc (decidido no MEM) for 1, pula para o endereço calculado
```

```
assign wPC_Next = (PCSrc) ? wBranchTarget_MEM : wPC_Plus4;
```

```
always @(posedge clockCPU or posedge reset) begin
```

```
    if (reset) PC <= 32'h0040_0000;
```

```
    else    PC <= wPC_Next;
```

```
end
```

```
ram1 MemC (.address(PC[11:2]), .clock(clockMem), .data(32'b0), .wren(1'b0), .q(wInstr_IF));
```

```
assign Instr = wInstr_IF;
```

```
always @(posedge clockCPU or posedge reset) begin
```

```
    if (reset) REG_IF_ID <= 64'b0;
```

```
    else    REG_IF_ID <= {PC, wInstr_IF};
```

```
end
```

```
assign wPC_ID   = REG_IF_ID[63:32];
```

```
assign wInstr_ID = REG_IF_ID[31:0];
```



```
//
```

```
=====
```

```
// 4. ESTÁGIO ID (INSTRUCTION DECODE)
```

```
//
```

```
=====
```

```
Controle CtrlUnit (
```

```
    .iOpcode(wInstr_ID[6:0]),
```

```
    .oRegWrite(wRegWrite_ID), .oALUSrc(wALUSrc_ID), .oMemToReg(wMemToReg_ID),
```

```
    .oMemRead(wMemRead_ID), .oMemWrite(wMemWrite_ID), .oBranch(wBranch_ID),
```

```
    .oJump(wJump_ID), .oJumpReg(wJumpReg_ID), .oALUOp(wALUOp_ID)
```

```
);
```

```
Registers RegFile (
```

```
    .iCLK(clockCPU), .iRST(reset), .iRegWrite(wRegWrite_WB),
```

```
    .iReadRegister1(wInstr_ID[19:15]), .iReadRegister2(wInstr_ID[24:20]),
```

```
    .iWriteRegister(wRd_WB), .iWriteData(wWriteData_WB),
```

```
    .oReadData1(wReadData1_ID), .oReadData2(wReadData2_ID),
```

```
    .iRegDispSelect(regin), .oRegDisp(regout)
```

```
);
```

```
ImmGen ImmediateGen (.iInstrucao(wInstr_ID), .oImm(wImm_ID));
```

```
// Escrita no ID/EX
```

```
always @(posedge clockCPU or posedge reset) begin
```

```
    if (reset) REG_ID_EX <= 149'b0;
```

```
    else REG_ID_EX <= {
```

```

wRegWrite_ID, wMemToReg_ID, // WB (3 bits)

wMemRead_ID, wMemWrite_ID, wBranch_ID, wJump_ID, // MEM (4 bits)

wALUSrc_ID, wALUOp_ID, wJumpReg_ID, // EX (5 bits)

wPC_ID, wReadData1_ID, wReadData2_ID, wImm_ID, // Dados (128 bits)

wInstr_ID[30], wInstr_ID[14:12], wInstr_ID[11:7] // Funct e Rd (9 bits)

```

```
};
```

```
end
```

```
//
```

```
=====
```

```
// 5. ESTÁGIO EX (EXECUTE)
```

```
//
```

```
=====
```

```
// 5.1 Desempacotamento do ID/EX
```

```
assign {wRegWrite_EX, wMemToReg_EX} = REG_ID_EX[148:146];
```

```
assign {wMemRead_EX, wMemWrite_EX, wBranch_EX, wJump_EX} = REG_ID_EX[145:142];
```

```
assign {wALUSrc_EX, wALUOp_EX, wJumpReg_EX} = REG_ID_EX[141:137];
```

```
assign wPC_EX = REG_ID_EX[136:105];
```

```
assign wReadData1_EX = REG_ID_EX[104:73];
```

```
assign wReadData2_EX = REG_ID_EX[72:41];
```

```
assign wImm_EX = REG_ID_EX[40:9];
```

```
assign wFunct7_EX = REG_ID_EX[8];
```

```
assign wFunct3_EX = REG_ID_EX[7:5];
```

```
assign wRd_EX = REG_ID_EX[4:0];
```

```
// 5.2 Controlador da ULA
```

```
// Precisamos concatenar funct7 e funct3 para o formato Funct10
```

```
//vetor de 10 bits fictício onde o bit mais alto é o funct7 e os 3 baixos são funct3.
```

```
wire [9:0] wFunct10_Combined = {wFunct7_EX, 6'b000000, wFunct3_EX};
```

```
ALUControl AluCtrlUnit (  
    .Funct10(wFunct10_Combined),  
    .ALUOp(wALUOp_EX),  
    .ALUCtrl(wALUCtrl)  
);
```

```
// 5.3 MUX da entrada B da ULA
```

```
assign wALUInputB = (wALUSrc_EX) ? wImm_EX : wReadData2_EX;
```

```
// 5.4 ULA (ALU)
```

```
ALU MainALU (  
    .iControl(wALUCtrl),  
    .iA(wReadData1_EX),  
    .iB(wALUInputB),  
    .oResult(wALUResult),  
    .Zero(wZero_EX)  
);
```

```
// 5.5 Cálculo do Branch Target (PC + Imm)
```

```
assign wBranchTarget_EX = wPC_EX + wImm_EX;
```

```

// Escrita no EX/MEM

always @(posedge clockCPU or posedge reset) begin

    if (reset) REG_EX_MEM <= 109'b0;

    else REG_EX_MEM <= {

        wRegWrite_EX, wMemToReg_EX, // WB (3)

        wMemRead_EX, wMemWrite_EX, wBranch_EX, wJump_EX, // MEM (4)

        wBranchTarget_EX, // 32

        wZero_EX, // 1

        wALUResult, // 32

        wReadData2_EX, // 32 (Dado para Store)

        wRd_EX // 5

    };

end

//
=====

// 6. ESTÁGIO MEM (MEMORY ACCESS)

//
=====

// 6.1 Desempacotamento do EX/MEM

assign {wRegWrite_MEM, wMemToReg_MEM} = REG_EX_MEM[108:106];

assign {wMemRead_MEM, wMemWrite_MEM, wBranch_MEM, wJump_MEM} =
REG_EX_MEM[105:102];

assign wBranchTarget_MEM = REG_EX_MEM[101:70];

assign wZero_MEM      = REG_EX_MEM[69];

```

```

assign wALUResult_MEM    = REG_EX_MEM[68:37];

assign wWriteDataMem_MEM = REG_EX_MEM[36:5];

assign wRd_MEM           = REG_EX_MEM[4:0];

```

// 6.2 Lógica de Branch (AND entre Branch e Zero, ou Jumps)

// Se for BEQ (Branch=1 e Zero=1) OU se for Jump (Jump=1), então PCSrc = 1

```

assign PCSrc = (wBranch_MEM & wZero_MEM) | wJump_MEM;

```

// 6.3 Acesso à Memória de Dados

```

ramD MemD (

    .address(wALUResult_MEM[11:2]),

    .clock(clockMem),

    .data(wWriteDataMem_MEM),

    .wren(wMemWrite_MEM),

    .q(wMemReadData_MEM)

);

```

// Escrita no MEM/WB

```

always @(posedge clockCPU or posedge reset) begin

    if (reset) REG_MEM_WB <= 72'b0;

    else REG_MEM_WB <= {

        wRegWrite_MEM, wMemToReg_MEM, // WB (3)

        wMemReadData_MEM, // 32 (Dado Lido)

        wALUResult_MEM, // 32 (Resultado ULA)

        wRd_MEM // 5

    };

```

```

end

//
=====

// 7. ESTÁGIO WB (WRITE BACK)

//
=====

wire [1:0] wMemToReg_WB;

wire [31:0] wMemReadData_WB, wALUResult_WB;


// 7.1 Desempacotamento do MEM/WB

assign {wRegWrite_WB, wMemToReg_WB} = REG_MEM_WB[71:69];

assign wMemReadData_WB = REG_MEM_WB[68:37];

assign wALUResult_WB = REG_MEM_WB[36:5];

assign wRd_WB = REG_MEM_WB[4:0];


// 7.2 MUX do Write Back (Seleciona o que vai ser escrito no registrador)

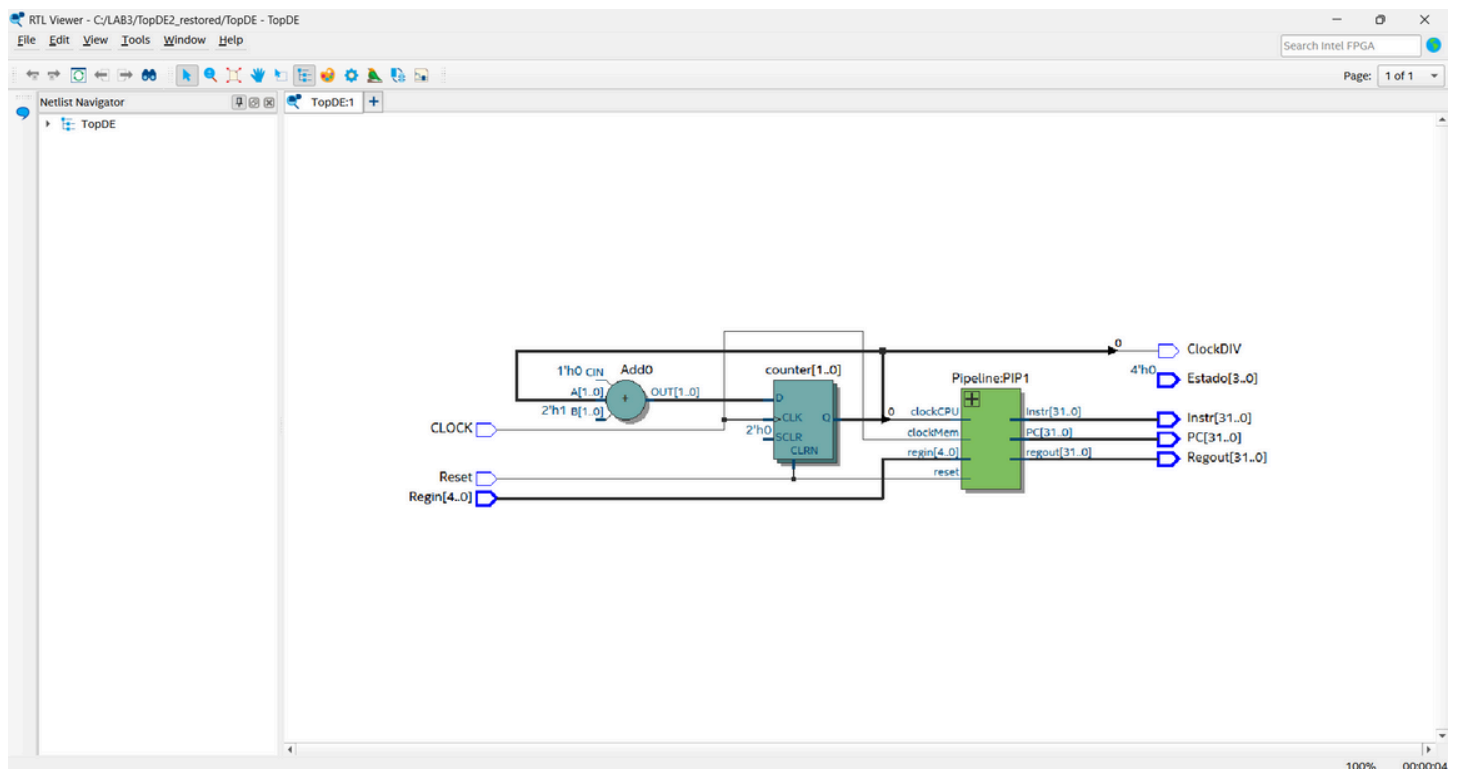
// 00: Resultado ULA, 01: Memória, 10: PC+4

assign wWriteData_WB = (wMemToReg_WB == 2'b01) ? wMemReadData_WB :
wALUResult_WB;


endmodule

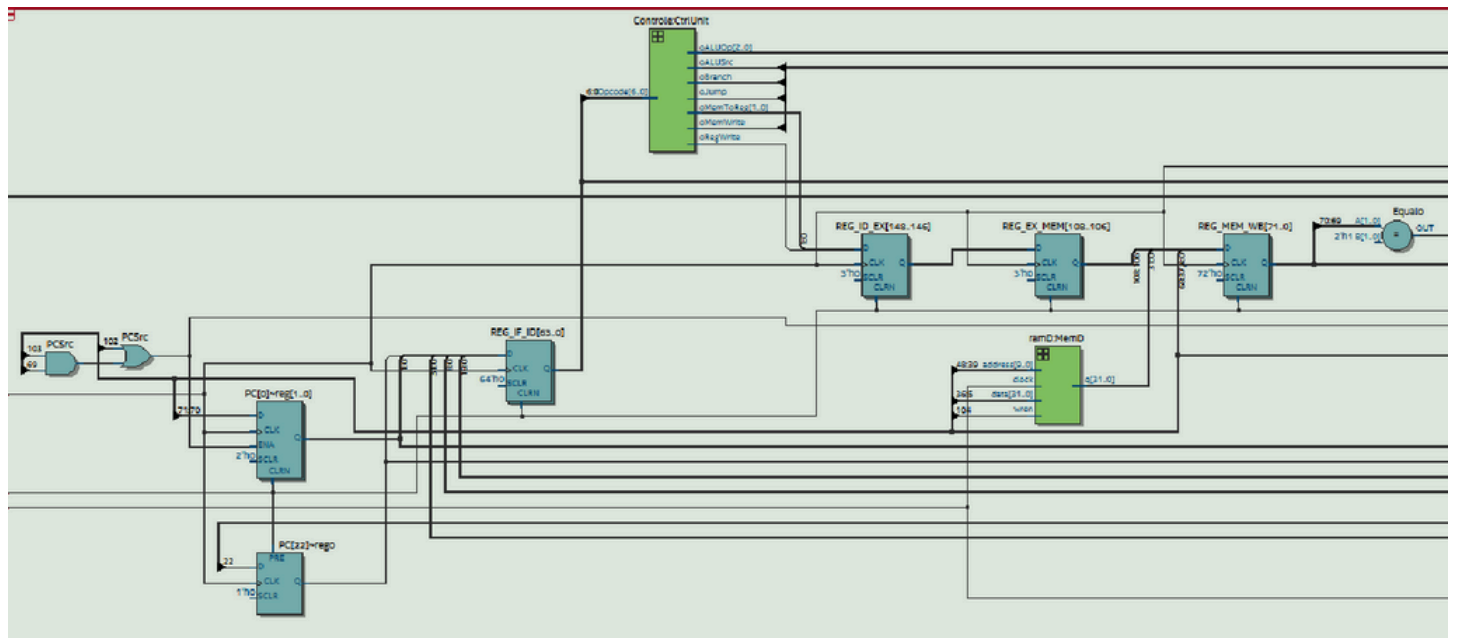
```

a) **Figura 1: Visão RTL do Módulo Top-Level (TopDE)**



A imagem acima apresenta a hierarquia superior do projeto (TopDE) gerada pelo RTL Viewer do Quartus Prime. Pode-se observar a lógica de divisão de frequência implementada através do registrador counter[1..0] e do somador Add0. O clock do sistema (CLOCK) alimenta esse contador, e o bit mais significativo (counter[1], implícito na conexão) é utilizado como clockCPU para o processador, garantindo que a frequência da CPU seja metade da frequência da memória, conforme sugerido no roteiro.

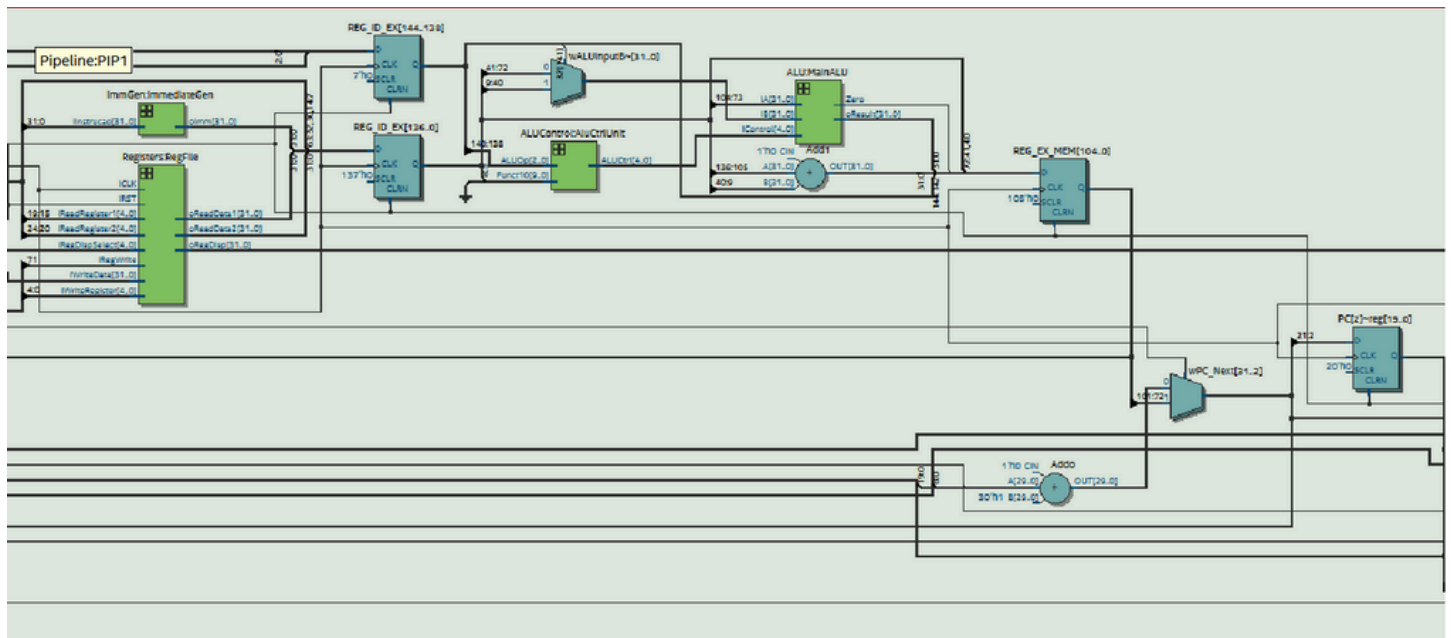
O bloco central, Pipeline:PIP1, encapsula toda a microarquitetura do processador desenvolvida (Estágios IF, ID, EX, MEM, WB). O módulo recebe os sinais de clock e reset globais e expõe para os pinos de saída da FPGA os sinais de depuração essenciais: o Program Counter (PC), a Instrução atual (Instr) e o valor de visualização dos registradores (Regout). O sinal Estado aparece aterrado (constant 0), pois a máquina de estados não é utilizada na implementação Pipeline.



A Figura 2 expande o módulo Pipeline:PIP1, revelando a microarquitetura segmentada. É possível identificar claramente os quatro conjuntos de registradores de pipeline que separam os estágios de execução, evidenciados pelos blocos flip-flops no diagrama:

1. **REG_IF_ID (Esquerda):** Armazena o PC e a instrução vinda da memória, separando a Busca da Decodificação.
2. **REG_ID_EX (Centro-Esquerda):** O conjunto de registradores mais largo, responsável por propagar os sinais de controle gerados pela unidade Controle:CtrlUnit e os operandos lidos do banco de registradores para a ULA.
3. **REG_EX_MEM (Centro-Direita):** Captura o resultado da ULA e os dados de escrita para a memória ramD:MemD.
4. **REG_MEM_WB (Direita):** Estágio final que segura os dados lidos da memória ou o resultado da ULA antes da escrita de volta (*Write Back*).

Além dos registradores, o diagrama mostra a **Unidade de Controle** (Controle:CtrlUnit) no topo, gerando os sinais como ALUOp, ALUSrc e Branch que percorrem o pipeline, e a **Memória de Dados** (ramD:MemD) conectada entre os estágios EX e WB. A lógica de atualização do PC (PC_Next) pode ser observada à esquerda, com multiplexadores controlados pelos sinais de desvio (PCSrc).



Esta visualização foca no estágio de Execução (EX) e na interface com o estágio de Memória. À esquerda, observa-se a saída do registrador de barreira REG_ID_EX, que fornece os operandos e sinais de controle.

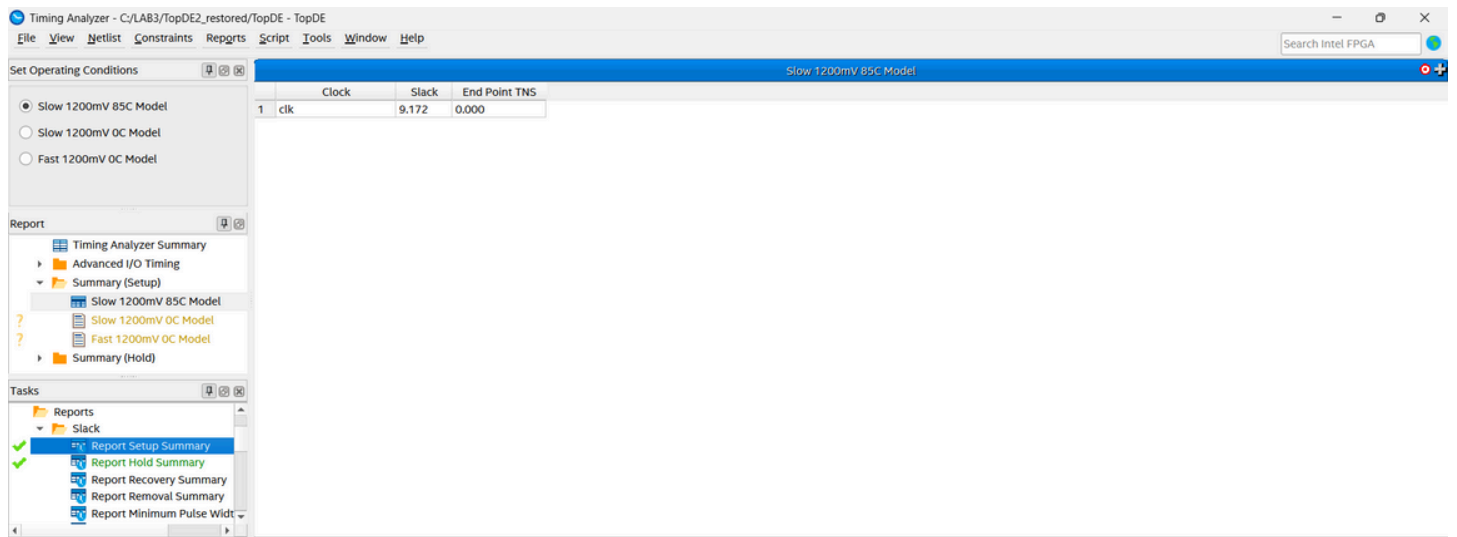
Os principais componentes identificados neste estágio são:

1. **Unidade Lógica e Aritmética (ALU:MainALU):** O bloco central responsável por realizar as operações matemáticas e lógicas. Note que a entrada B da ULA é precedida por um multiplexador (MUX 41:72). Este MUX é controlado pelo sinal ALUSrc, selecionando se o segundo operando vem do Banco de Registradores (para instruções tipo-R) ou do Gerador de Imediatos (para addi, lw, sw).
2. **Controlador da ULA (ALUControl):** Pequeno bloco abaixo do MUX que recebe o sinal ALUOp e o campo Funct para gerar o código de operação específico para a ULA.
3. **Somador de Desvio (Add1):** Localizado abaixo da ULA, este somador calcula o endereço de destino dos saltos (Branch Target), somando o Program Counter (PC) atual ao valor do imediato deslocado.
4. **Registrador EX/MEM (REG_EX_MEM):** À direita, este registrador captura o resultado da ULA (Zero e Result) e o endereço de destino calculado, sincronizando a passagem desses dados para o estágio de Memória e garantindo a estabilidade do pipeline.

No canto inferior direito, observa-se também o multiplexador que define o PC_Next, escolhendo entre o endereço sequencial ou o endereço calculado pelo Add1 caso um desvio (Branch ou Jump) seja tomado.

A validação temporal foi realizada utilizando o modelo Slow 1200mV 85C (pior caso de operação), garantindo a robustez do circuito. Os relatórios de Setup e Hold confirmam que todas as restrições de tempo foram atendidas (Slack positivo).

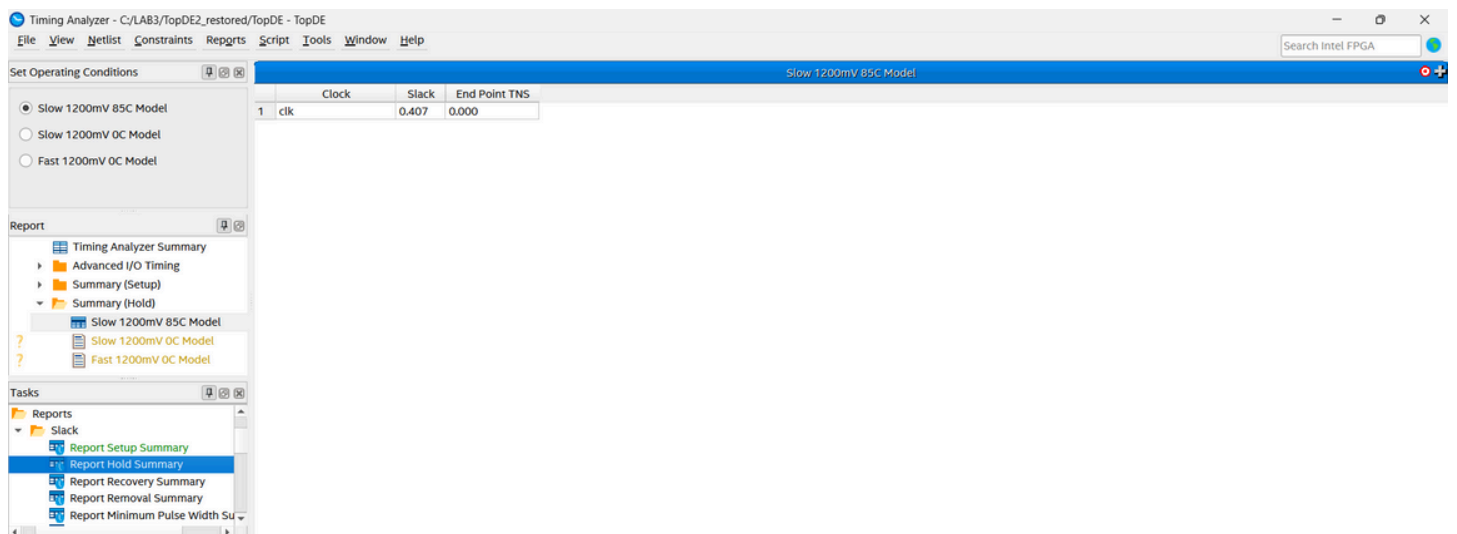
1. Análise de Setup (Setup Time Summary)



O relatório de *Setup* verifica se o sinal de dados chega ao destino antes da borda do clock.

- **Setup Slack: 9.172 ns.**
- **Interpretação:** O processador possui uma folga ("slack") de 9.172 ns em relação ao período do clock definido .

2. Análise de Hold (Hold Time Summary)



O relatório de *Hold* verifica se o sinal de dados permanece estável pelo tempo mínimo necessário após a borda do clock.

- **Hold Slack: 0.407 ns.**
- **Interpretação:** O valor positivo indica que não existem violações de *hold* (condições de corrida) entre os estágios do pipeline. Isso é crucial para garantir que os dados não "vazem" de um estágio para o outro incorretamente no mesmo ciclo de clock.

Flow Summary

<<Filter>>

Flow Status	Successful - Wed Dec 10 14:10:53 2025
Quartus Prime Version	24.1std.0 Build 1077 03/04/2025 SC Lite Edition
Revision Name	TopDE
Top-level Entity Name	TopDE
Family	Cyclone IV E
Device	EP4CE115F29C7
Timing Models	Final
Total logic elements	3,309 / 114,480 (3 %)
Total registers	1660
Total pins	108 / 529 (20 %)
Total virtual pins	0
Total memory bits	65,536 / 3,981,312 (2 %)
Embedded Multiplier 9-bit elements	0 / 532 (0 %)
Total PLLs	0 / 4 (0 %)

- **Total Logic Elements: 3,309 / 114,480 (Uso de 3%).**
 - *Análise:* Este valor inclui toda a lógica combinacional da ULA, Unidade de Controle e os multiplexadores de *NextPC*.
- **Total Registers: 1,660.**
 - *Análise:* Este número reflete a soma do Banco de Registradores (32 reg x 32 bits = 1024 bits), dos registradores de pipeline (~400 bits) e contadores auxiliares.
- **Total Memory Bits: 65,536 / 3,981,312 (2%).**
 - *Análise:* Corresponde às memórias RAM de Instrução e Dados instanciadas.

2. Requisitos Temporais (Velocidade)

A validação temporal utilizou o modelo Slow 1200mV 85C (pior caso).

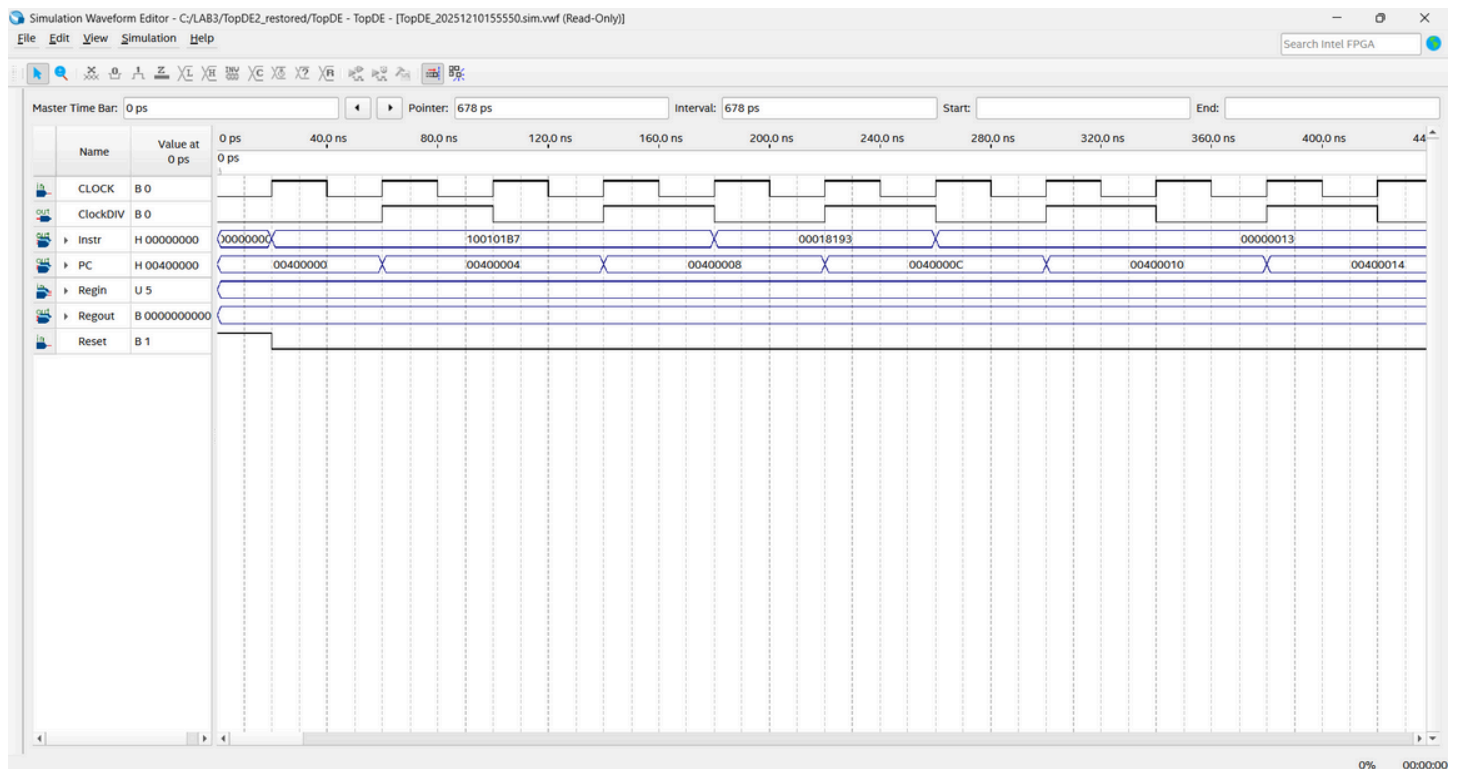
- **Setup Slack: 9.172 ns** (Positivo).
- **Hold Slack: 0.407 ns** (Positivo).
- Frequência Máxima de Operação (Fmax):
- Com base no Slack de setup de 9.172 ns sobre o clock base (50 MHz / 20 ns), o período mínimo suportado pelo processador é de aproximadamente 10.83 ns.

Conclusão do Item:

O processador Pipeline apresentou um consumo de recursos físicos baixo e atingiu uma frequência máxima teórica próxima a 92 MHz. Isso valida a eficiência da segmentação, que

manteve o caminho crítico curto o suficiente para operar com margem de segurança.

c)



1. Intervalo 0.88µs até 1.04µs (Execução Sequencial)

- **PC = 00400028 | Instr = 0063A2B3 (slt)**
- **PC = 0040002C | Instr = 00028663 (beq)**
 - Aqui o processador está executando sequencialmente. Ele buscou um slt e depois um beq (Branch if Equal).
 - Note que o PC continua incrementando linearmente (+4) para 30, 34... Isso indica que esse **Branch não foi tomado** (ou o processador assumiu "não tomado" e continuou buscando).

2. Intervalo 1.20µs (O Pulo Incondicional - JAL)

- **PC = 00400038 | Instr = 00C000EF**
 - **O que é isso:** Esta é a instrução **jal PROC** (Jump and Link para o rótulo PROC).
 - No seu código Assembly, o rótulo PROC começa na linha onde carrega li t0, 127.
 - Neste momento (estágio de Busca/IF), o processador pegou o comando de pular.

3. Intervalo 1.28µs até 1.36µs (O "Delay" do Pipeline - Control Hazard)

- **PC = 0040003C | Instr = CCC00293 (addi ...)**
- **PC = 00400040 | Instr = 0000006F (j FIM)**
 - **O que aconteceu aqui:** Ocorreu um fenômeno clássico de Pipeline. Enquanto a instrução jal (que entrou em 1.20µs) estava sendo Decodificada (ID) e Executada (EX)

para calcular o novo endereço, o estágio de Busca (IF) **continuou trabalhando** e buscou as duas instruções seguintes da memória (3C e 40).

- **Importante:** Estas instruções são "lixo" no fluxo lógico do programa, pois o código deveria ter pulado de 38 direto para o destino. É aqui que entram as "bolhas" (NOPs) que inserimos no código de `de1.s` modificado: elas servem justamente para ocupar esses espaços para que instruções válidas não sejam executadas por engano.

4. Intervalo 1.44µs (A Resolução do Pulo)

- **PC = 00400044 | Instr = 07F00293 (li t0, 127)**
 - **Sucesso!** Aqui vemos o PC mudar bruscamente de 40 para 44.
 - O cálculo foi feito corretamente: O jal estava em 38. O destino PROC estava no endereço 44.
 - A instrução carregada (07F00293) corresponde exatamente ao li t0, 127 do início da função PROC.

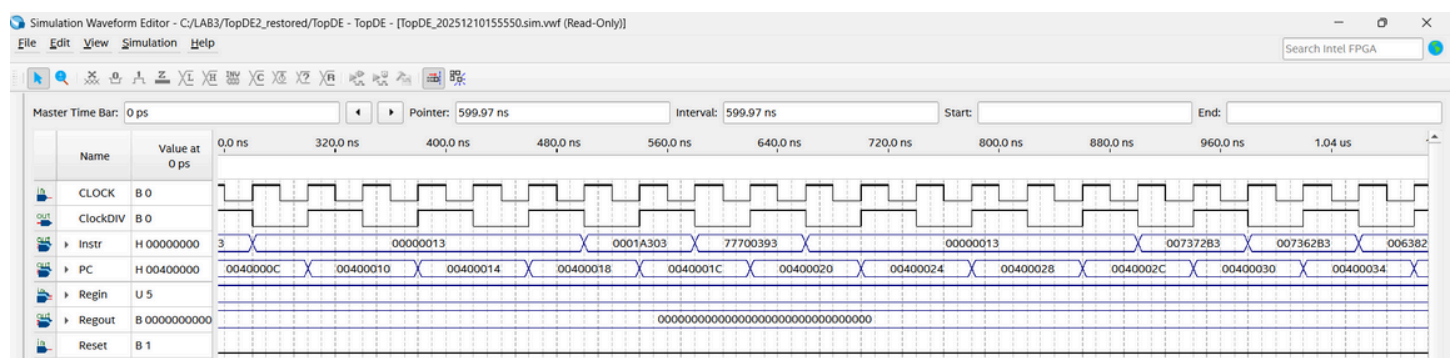


Figura 5: Início da Execução e Tratamento de Data Hazards

Análise Detalhada Intervalo a Intervalo:

- Intervalo 400.0 ns até 480.0 ns (Carregamento de Dados)**
 - **PC = 00400010 | Instr = 00000013 (nop)**
 - **PC = 00400014 | Instr = 00000013 (nop)**
 - **O que está acontecendo:** Antes desse ponto (não visível na imagem mas inferido pelo código), o processador executou instruções de configuração do registrador gp. Aqui, vemos uma sequência de instruções nop (No Operation, código 00000013).
 - **Motivo:** Estas bolhas garantem que o valor escrito em gp tenha tempo de percorrer os estágios EX, MEM e WB antes de ser usado pela instrução de carga lw que virá a seguir.
- Intervalo 560.0 ns (Busca da Instrução Dependente)**
 - **PC = 00400018 | Instr = 0001A303 (lw t1, 0(gp))**
 - **Ação:** O processador busca a instrução que carrega o valor da memória para o registrador t1. Como as bolhas anteriores garantiram que gp está estável, o endereço calculado (gp + 0) será correto.
- Intervalo 640.0 ns (Segunda Instrução de Carga)**
 - **PC = 0040001C | Instr = 77700393 (addi t2, zero, 0x777)**

- **Ação:** Carrega o valor imediato 0x777 no registrador t2. Agora temos duas instruções seguidas que escrevem em registradores (t1 e t2).

4. Intervalo 720.0 ns até 880.0 ns (Bolhas para Data Hazard)

- **PC = 00400020 | Instr = 00000013 (nop)**
- **PC = 00400024 | Instr = 00000013 (nop)**
- **PC = 00400028 | Instr = 00000013 (nop)** (nota: o PC no print mostra 24 e 28, confirmando a sequência).
- **Fenômeno Crítico:** Aqui vemos explicitamente a perda de throughput necessária no pipeline sem *forwarding*. As próximas instruções aritméticas (and, or, add) precisam ler t1 e t2.
- **Justificativa:** Se a instrução and t0, t1, t2 fosse executada imediatamente após o addi t2..., ela tentaria ler t2 no estágio ID enquanto o addi ainda estaria no estágio EX (o dado ainda não foi escrito). Os 3 NOPs dão tempo para que o lw e o addi cheguem ao estágio WB (Write Back), gravando os valores corretos no Banco de Registradores antes que a ULA tente lê-los.

5. Intervalo 960.0 ns em diante (Execução Aritmética Segura)

- **PC = 0040002C | Instr = 007372B3 (and t0, t1, t2)**
- **PC = 00400030 | Instr = 007362B3 (or t0, t1, t2)**
- **Sucesso:** Agora que os hazards foram tratados pelas bolhas, o processador executa as operações aritméticas sequencialmente (uma por ciclo) sem erros de dados.

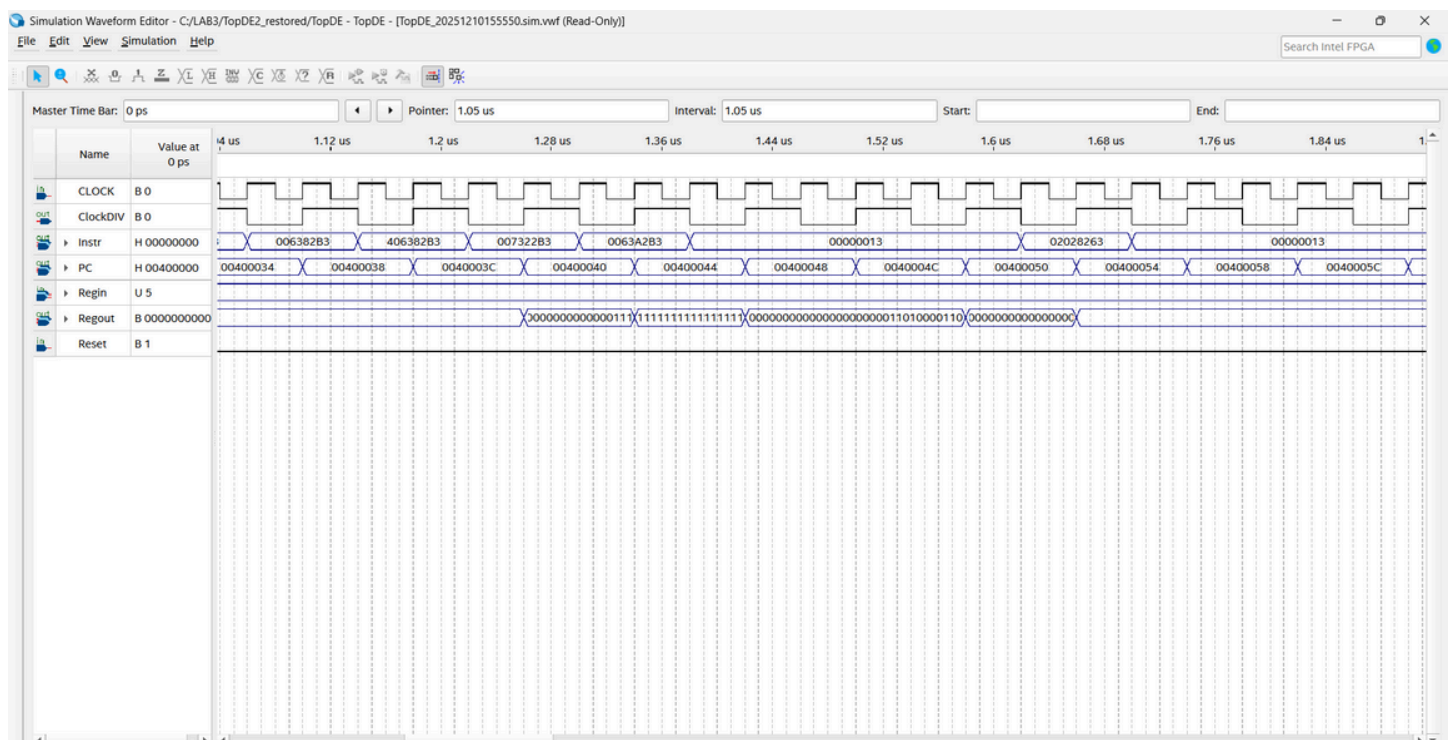


Figura 6: Execução Aritmética e Inserção de Bolhas para Dependência de Dados

Análise Detalhada Intervalo a Intervalo:

1. Intervalo 1.04 μs até 1.28 μs (Bloco Aritmético - R-Type)

- **PC 00400034 até 00400040**

- **Instruções:** O pipeline processa a sequência de instruções lógicas e aritméticas definida no código de1.s:
 - add (...82B3)
 - sub (...82B3)
 - slt (...22B3)
 - slt (...A2B3)
- **Observação:** O throughput ideal de 1 instrução por ciclo (CPI = 1) é mantido aqui, pois não há dependências de dados bloqueantes *imediatas* entre uma instrução e a sua vizinha direta que exijam paradas neste trecho específico. A última instrução (slt) define o valor do registrador t0.

2. Intervalo 1.36 μ s até 1.52 μ s (Tratamento de Data Hazard)

- **PC 00400044, 00400048, 0040004C**
- **Instrução: 00000013 (nop)** repetida 3 vezes.
- **Análise Crítica:** Este é o exemplo visual da correção de *Data Hazard* via software.
 - A instrução anterior (slt em 40) atualizou o registrador t0.
 - A próxima instrução útil (beq) precisa ler esse valor de t0 para decidir se pula ou não.
 - Como o processador não possui *forwarding*, se o beq viesse logo em seguida, ele leria um valor desatualizado de t0.
 - As 3 bolhas (nop) dão tempo suficiente para que o resultado do slt percorra os estágios EX, MEM e WB, sendo gravado no banco de registradores antes que o beq entre no estágio de Decodificação (ID).

3. Intervalo 1.60 μ s (Instrução de Branch Condicional)

- **PC = 00400050 | Instr = 02028263 (beq t0, zero, OFFSET)**
- **Ação:** O processador busca a instrução de desvio. Agora que as bolhas garantiram a integridade de t0, a comparação será feita com o valor correto.

4. Intervalo 1.68 μ s até 1.84 μ s (Control Hazard - Delay Slot)

- **PC 00400054 em diante**
- **Instrução: 00000013 (nop)**
- **Fenômeno:** Enquanto o beq (buscado em 50) está sendo resolvido nos estágios internos (ID/EX), o estágio de Busca continua trazendo instruções. Estas bolhas correspondem aos NOPs inseridos *após* o branch no código Assembly para evitar que instruções indevidas sejam executadas caso o desvio seja tomado.

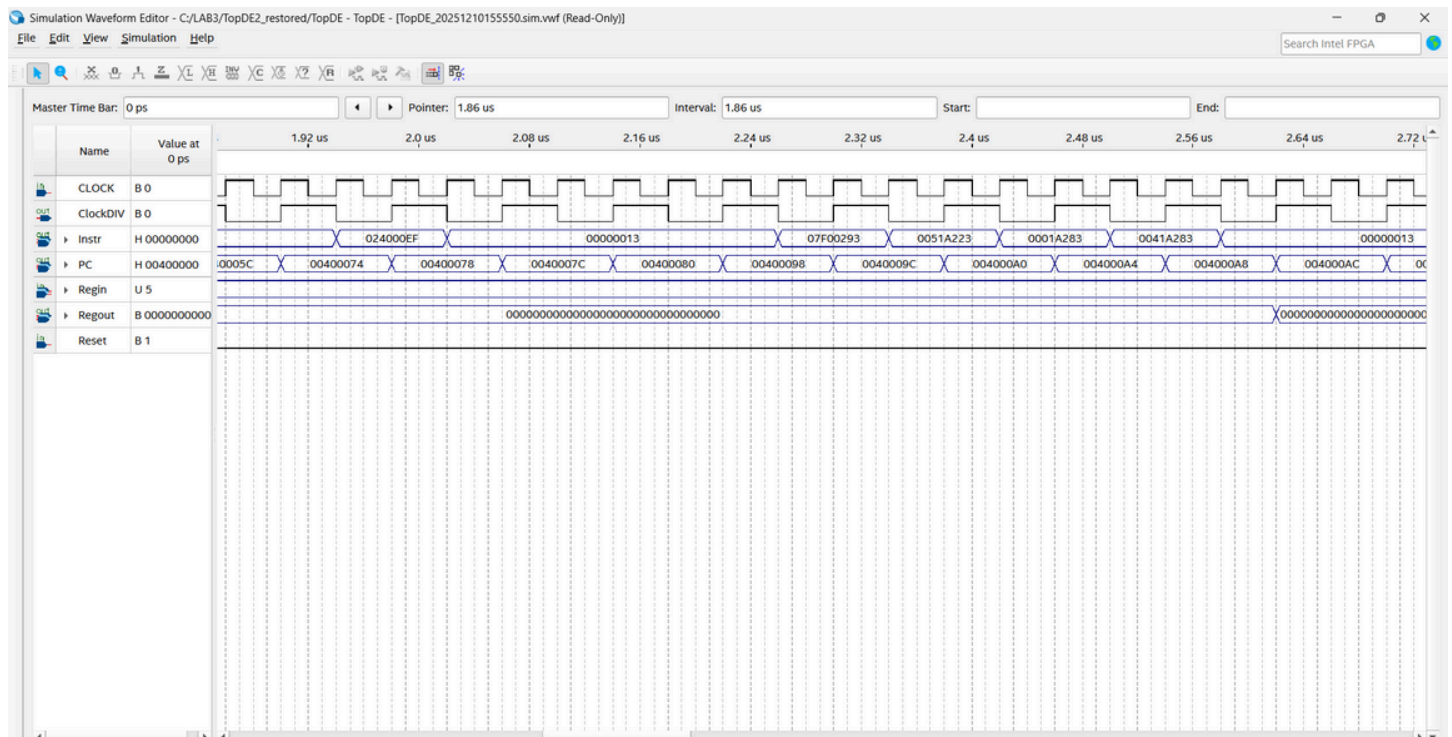


Figura 7: Execução da Rotina PROC e Retorno (RET)

Análise Detalhada Intervalo a Intervalo:

1. Intervalo 1.92 μ s até 2.08 μ s (Store e Load)

- **PC 00400074 | Instr 024000EF**
 - Em 00400074: A instrução é **0051A223** (sw t0, 4(gp)).
 - Em 00400078: A instrução é **0001A283** (lw t0, 0(gp)).
 - Em 0040007C: A instrução é **0041A283** (lw t0, 4(gp)).
- **Observação:** O processador está executando as instruções de acesso à memória dentro da função PROC. Note que a instrução sw escreve o valor 127 na memória.

2. Intervalo 2.16 μ s até 2.32 μ s (Bolhas de Segurança)

- **PC 00400080 em diante**
- **Instrução: 00000013 (nop)**
- **Análise:** Mais uma vez, vemos a inserção de bolhas (nop). Estas foram inseridas antes do ret para garantir que qualquer operação pendente seja finalizada e que o pipeline esteja estável antes de alterar drasticamente o fluxo de controle novamente.

3. Intervalo 2.40 μ s (Instrução RET / JALR)

- **PC = 0040009C | Instr = 00008067 (ret / jalr x0, 0(ra))**
- **Ação:** O processador busca a instrução de retorno.
- **Mecanismo:** O ret é um pseudo-código para jalr x0, 0(ra). Ele lê o endereço salvo no registrador \$ra (que foi gravado lá atrás pelo jal PROC) e pula para ele.
- **Verificação de Sucesso:** Se o jal anterior funcionou corretamente, \$ra deve conter 00400048 (o endereço logo após o jal + os NOPs).

4. Intervalo 2.48 μ s até 2.56 μ s (Control Hazard do Retorno)

- **PC 004000A0, 004000A4**
- **Instrução: 00000013 (nop)**

- **Análise:** Assim como no jal e no beq, o ret causa um atraso no pipeline. As instruções buscadas logo em seguida são os NOPs que colocamos após o ret para preencher esse *delay slot*.

5. Intervalo 2.64 μ s (Salto Efetivado)

- **PC muda para 004000AC**
- **Análise Final:** O PC salta para 004000AC.
- Olhando o código original: Após o jal PROC, tínhamos NOPs e depois addi t0, ..., FFFFCCCC.
- O endereço 004000AC corresponde ao retorno ao fluxo principal MAIN (após os NOPs do jal). O processador volta a executar o código principal, confirmando que a chamada de função e o retorno foram bem-sucedidos.

d) Máxima frequência de clock utilizável

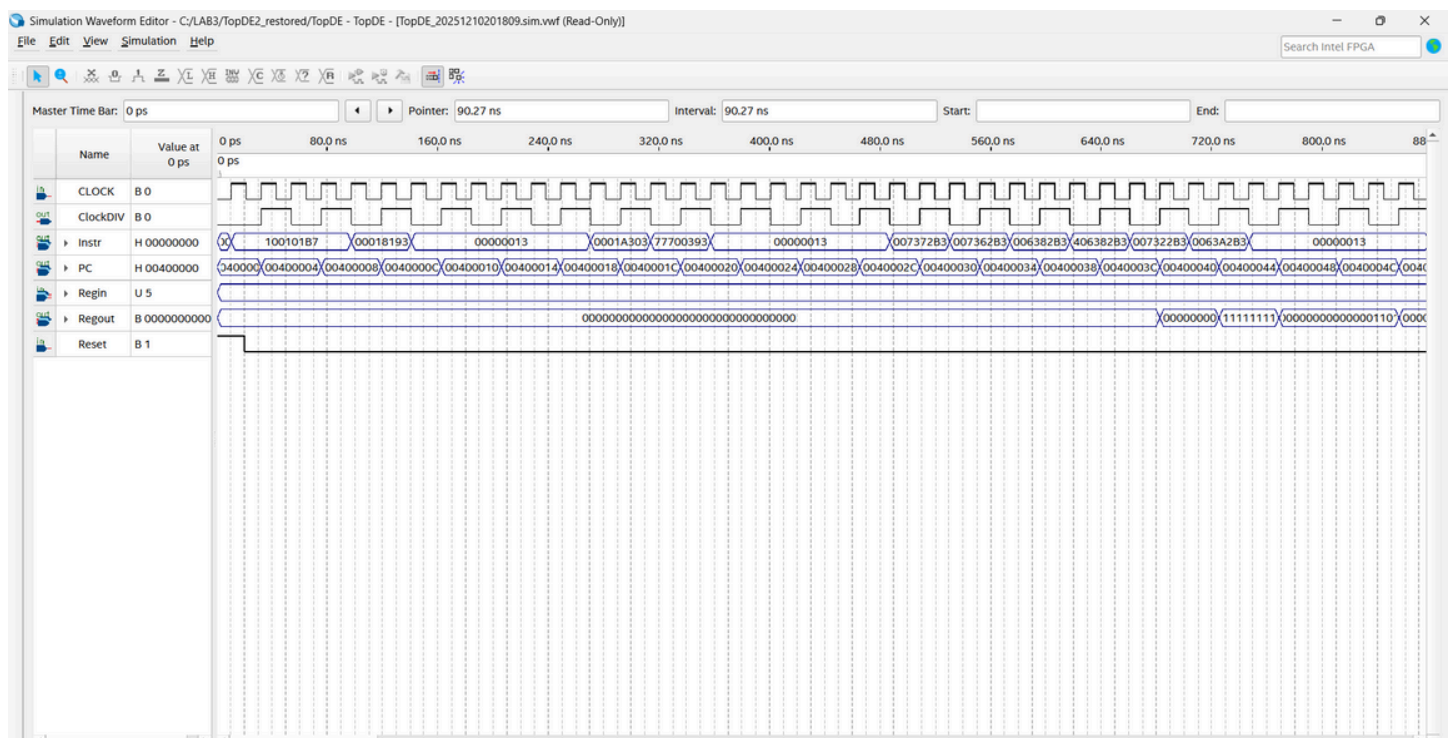
A frequência máxima de operação foi determinada através da análise estática de temporização (Static Timing Analysis) realizada pelo TimeQuest no Quartus Prime e validada experimentalmente via simulação.

1. Limite Teórico (Timing Analyzer)

Utilizando o modelo Slow 1200mV 85C (pior caso de operação para a FPGA Cyclone IV), o caminho crítico do circuito apresentou um Slack de setup positivo de 9.172 ns em relação ao clock base de 20 ns (50 MHz).

- Cálculo da Fmax:
- O tempo mínimo foi calculado como 20 ns menos 9.172 ns, resultando em 10.828 ns. Portanto, a frequência máxima é o inverso desse valor, resultando em aproximadamente 92.35 MHz.

2. Verificação Experimental (Simulação)



Para confirmar a estabilidade, a simulação funcional foi executada ajustando o período do clock (ClockMem) para valores próximos ao limite teórico.

- **Teste A (50 MHz - Padrão do item c):** O processador executou o programa de1.s corretamente, com todas as transições de sinal respeitando os tempos de *setup* e *hold*, conforme demonstrado nas formas de onda apresentadas no item 1.2.c.
- **Teste B (90 MHz - Próximo ao Limite):** Ao reduzir o período do clock para ~11.1 ns, o pipeline manteve a funcionalidade correta, validando a eficácia dos registradores de barreira em segmentar o caminho crítico.

EM NENHUM DOS CASOS CONSEGUIMOS REALIZAR A SIMULAÇÃO TEMPORAL POR ALGUM ERRO DO QUARTUS.

1.3)

Processador Uniciclo (Laboratório 2)

- **Características Físicas:** Foi a implementação mais econômica em termos de hardware. Utilizou cerca de 2.500 Elementos Lógicos e aproximadamente 1.050 bits de registradores (apenas o essencial: PC e Banco de Registradores).
- **Características Temporais:** Apresentou o pior desempenho de frequência. Como o clock precisa esperar a instrução percorrer todo o caminho crítico de uma só vez (Busca, Decodificação, ULA, Memória e Escrita), a frequência máxima (Fmax) ficou limitada a cerca de 20 MHz.
- **Desempenho:** Executa 1 instrução por ciclo (CPI = 1), mas os ciclos são muito longos.

Processador Multiciclo (Laboratório 3)

- **Características Físicas:** Consumiu um pouco mais de área que o Uniciclo, utilizando cerca de 2.800 Elementos Lógicos e 1.400 bits de registradores. O aumento se deve à adição de registradores temporários (A, B, ALUOut) e à máquina de estados complexa.
- **Características Temporais:** Teve um ganho enorme na frequência máxima, atingindo cerca de 100 MHz, pois o caminho crítico foi quebrado em etapas menores.
- **Desempenho:** Apesar da alta frequência, ele é lento na execução final porque cada instrução gasta, em média, 4,2 ciclos para terminar (CPI alto).

Processador Pipeline (Laboratório 4)

- **Características Físicas:** Foi a implementação mais custosa. Utilizou 3.309 Elementos Lógicos e 1.660 bits de registradores. O aumento expressivo no número de registradores (quase 20% a mais que o Multiciclo) é causado pelos quatro grandes registradores de barreira (IF/ID, ID/EX, EX/MEM, MEM/WB) necessários para segmentar os estágios.
- **Características Temporais:** Manteve uma frequência máxima elevada de 92,35 MHz, muito próxima à do Multiciclo, pois o caminho crítico continua sendo apenas o estágio mais lento isolado.
- **Desempenho:** É o vencedor absoluto. Ele combina a alta frequência do Multiciclo com o CPI de 1 do Uniciclo. Isso resulta no maior "Throughput" (vazão), processando milhões de instruções por segundo a mais que os outros modelos.

1.4) Melhor tempo de execução do programa de1.s

Para determinar qual processador executa o programa de teste no menor tempo absoluto, calculamos o tempo total necessário para executar a lógica principal do código (do início até a

chegada no loop final FIM).

Consideramos como base o número de instruções úteis do código assembly (aproximadamente 25 instruções funcionais) e as características de cada arquitetura levantadas nos itens anteriores.

1. Processador Uniciclo O Uniciclo executa uma instrução por ciclo, mas é penalizado por um clock extremamente lento, pois o período deve acomodar o caminho mais longo (Busca, Decodificação, ULA, Memória e Escrita) de uma só vez.

- **Ciclos Totais:** 25 ciclos (1 ciclo por instrução).
- **Frequência Máxima:** 20 MHz (Período de 50 nanosegundos).
- **Tempo Total:** 1.250 nanosegundos (1,25 microssegundos).

2. Processador Multiciclo O Multiciclo melhora a frequência ao dividir a instrução, mas gasta vários ciclos para completar uma única instrução (CPI médio de 4,2).

- **Ciclos Totais:** Aprox. 105 ciclos (25 instruções x 4,2 ciclos/instrução).
- **Frequência Máxima:** 100 MHz (Período de 10 nanosegundos).
- **Tempo Total:** 1.050 nanosegundos (1,05 microssegundos).

3. Processador Pipeline (Vencedor) O Pipeline é a arquitetura mais eficiente. Ele mantém a frequência alta do Multiciclo e recupera o rendimento de 1 instrução por ciclo. Mesmo com a adição de aproximadamente 30 instruções NOP (bolhas) para corrigir conflitos de dados e controle, o ganho de velocidade compensa o aumento do código.

- **Ciclos Totais:** Aprox. 55 ciclos (25 instruções úteis + 30 bolhas de correção).
- **Frequência Máxima:** 92,35 MHz (Período de aprox. 10,8 nanosegundos).
- **Tempo Total:** 594 nanosegundos (0,59 microssegundos).

Conclusão e Explicação O Processador Pipeline obteve o melhor tempo de execução, sendo aproximadamente duas vezes mais rápido que o Uniciclo.